

Reviewer Pack — Fourier Coefficient Concentration and ACC⁰ Circuit Size

Entry 56779c9244c7 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-10 14:39:30 UTC

Fourier Coefficient Concentration and ACC⁰ Circuit Size

Entry ID: 56779c9244c7 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-10 14:39:30 UTC

1. Conjecture

Field A (mathematical branch): Fourier Analysis of Boolean Functions **Field B** (complexity object): ACC⁰ Circuit Size for 3-CNF Formulas

Statement:

For a 3-CNF formula with n variables, the sum of absolute Fourier coefficients is at least $\Omega(2^{\{n/2\}})$ if the formula cannot be computed by an ACC⁰ circuit of size $o(2^{\{n/2\}})$.

Rationale (proposer’s reasoning):

Fourier coefficients capture variable influence and correlation structure, which are critical for ACC⁰ lower bounds. High concentration in Fourier coefficients may indicate structured functions resistant to ACC⁰ computation.

Taxonomy category: FOURIER_ANALYTIC (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): ae631066bec9bcd5

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def generate_3cnf(n: int, m: int) -> list:
    clauses = []
    for _ in range(m):
        clause = set()
        while len(clause) < 3:
            var = random.randint(1, n)
            sign = random.choice([-1, 1])
            if (var, sign) not in clause and (-var, -sign) not in clause:
                clause.add((var, sign))
        clauses.append(list(clause))
    return clauses

def fast_walsh_hadamard_transform(f: list) -> list:
    n = len(f)
    while n > 1:
        half = n // 2
        for i in range(half):
            for j in range(i, n, n):
                u = f[j]
                v = f[j + half]
                f[j] = u + v
                f[j + half] = (u - v) * math.sqrt(2)
        n //= 2
    return f

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = 40
    m = 15 * n
    threshold = 2**(n/2) / (2**(n/2) // 10)

    f = [0] * (1 << n)
    for _ in range(m):
        clause = generate_3cnf(n, 1)[0]
        for var, sign in clause:
            f[1 << (var - 1)] += sign

    abs_sum = sum(abs(x) for x in f)

    conjecture_holds = abs_sum < threshold
    counterexample = "" if conjecture_holds else "threshold exceeded"
```

```

return {
    "metric_name": "sum_of_abs_fourier_coeffs",
    "metric_value": abs_sum,
    "instances_tested": 1,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or [2**i - 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{result['metric_name']}\", \"metric_
        results.append(result)

    mean = sum(r["metric_value"] for r in results) / len(results)
    std = math.sqrt(sum((r["metric_value"] - mean)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean} std={std} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"threshold exceeded\" first_failing_seed={first_
    else:
        print("RESULT: INCONCLUSIVE insufficient evidence")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_2c6c15b2.py", line 74, in <module>
    result = run_trial(seed)
             ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_2c6c15b2.py", line 49, in run_trial
    f = [0] * (1 << n)
         ~~~~~^~~~~~
MemoryError

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed with MemoryError, preventing data collection. Cannot assess conjecture validity without successful execution. | next: Optimize memory usage in the Fourier coefficient computation (e.g., use sparse representations or iterative methods)

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	109358
2	propose	ollama_remote	qwen3:8b	0	0	93696
3	preregistration	ollama_remote	qwen3:8b	0	0	24060
4	novelty	ollama_remote	qwen3:8b	0	0	22045
5	novelty	ollama_remote	qwen3:8b	0	0	15258
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13551
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8719
8	judge	ollama_remote	qwen3:8b	0	0	16090

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 302777 ms total latency. Provider mix: {'ollama_remote': 8}

(full prompt+response transcripts available in `research/audit/56779c9244c7.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/56779c9244c7.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/56779c9244c7.tar.gz` (if generated)